

Industrial Oriented Mini Project(CM653PC)
ON
Email and Website Threat Detection System

Submitted
In partial fulfillment of the requirements for the
award of the degree of

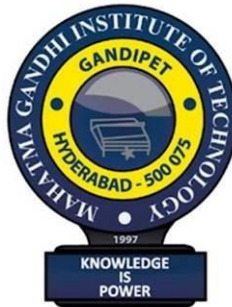
Bachelor of Technology
in
Computer Science Engineering
(Artificial Intelligence and Machine Learning)

by
D Sai Prakash (23261A6611)
&
Mekala VishnuVardhan(23261A6628)

Under the guidance of

Dr. Barnali Gupta Banik
(Associate Professor)

Dr. Shaik Irfan Babu
(Assistant Professor)



Department of Emerging Technologies
Mahatma Gandhi Institute of Technology (Autonomous)
(Affiliated to Jawaharlal Nehru Technological University Hyderabad)
Kokapet(V), Gandipet(M), Hyderabad. Telangana – 500075

2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
(Affiliated To Jawaharlal Nehru Technological University Hyderabad)

GANDIPET, HYDERABAD-500075, Telangana

CERTIFICATE



This is to certify that the project entitled —"Email and Website Threat Detection System" is being submitted by **D SAI PRAKASH (23261A6611)**, **MEKALA VISHNU VARDHAN (23261A6628)** in partial fulfillment of the requirements for the Industrial Oriented Mini Project (CM653PC) in **COMPUTER SCIENCE AND ENGINEERING (AI & ML)** is a record of bonafide work carried out by them under our guidance and supervision.

The results of the investigations enclosed in this report have been verified and found satisfactory.

PROJECT GUIDE

HEAD OF THE DEPARTMENT

Dr. Barnali Gupta Banik
Associate Professor
Department of ET

Dr. M. Rama Bai
Professor & Head
Department of ET

Dr. Shaik Irfan Babu
Assistant Professor
Department of ET

EXTERNAL EXAMINER

Declaration

This is to certify that the work reported in this project titled "**Email and Website Threat Detection System**" is a record of work done by us in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

D SAI PRAKASH

(23261A6611)

MEKALA VISHNUVARDHAN

(23261A6628)

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hard work and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We would like to express our sincere thanks to **Prof. G. Chandra Mohan Reddy, Principal MGIT**, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to **Prof. M. Rama Bai, Professor and HoD**, Department of Emerging Technologies, MGIT, for all the timely support and valuable suggestions during the period of the project.

We are extremely thankful and indebted to our mentors **Dr. Barnali Gupta Banik, Associate Professor, Dr. Shaik Irfan Babu, Assistant Professor**, Department of Emerging Technologies, for their constant guidance, encouragement and moral support throughout the project.

We are extremely thankful to **Ms. J. Sreedevi, Assistant Professor, Ms. Asha Jyothi, Assistant Professor**, Department of Emerging Technologies, MGIT, IOMP project coordinator for their encouragement and support throughout the project.

Finally, we would also like to thank all the faculty and staff of the Emerging Technologies Department who helped us directly or indirectly to complete this project.

D SAI PRAKASH

(23261A6611)

MEKALA VISHNUVARDHAN

(23261A6628)

Table of Contents

TOPIC	PAGENO.
Certificate	I
Declaration	II
Acknowledgement	III
List of Figures	VI
List of Tables	VII
Abstract	VIII
1 Introduction	1
1.1 Introduction to the Project	1
1.2 Existing Solution	1
1.3 Problem Definition	2
1.4 Proposed System	3
1.5 Requirements Specification	4
2 Literature Survey	6
3 Design and Methodology	10
3.1 System Architecture	
3.2 Technologies Used / Modules	10
3.3 System Components	11
3.4 Working Application	11
3.5 User Interface	12
3.6 Additional UX Enhancements	12
4 Implementation and Results	13
4.1 Overview	13
4.2 Front-End Components	13
4.3 Back-End Components	14
4.4 System Integration	15
4.5 Key Features Implemented	15
4.6 Sample Results	16
4.7 Result Analysis	17

5 Conclusion and Future Scope	20
References	21
Appendix	22

List of Figures

FIGURE NO.	FIGURE NAME	PAGENO.
3.1	System Flowchart — 5-layer pipeline from Input to Output	09
4.1	Gradio frontend showing Full Scan Tab with PHISHING verdict	18
4.2	Quick URL Tab showing GENUINE verdict for Indian government site	19
4.3	XAI Evidence Report showing SHAP and LIME explanations	19

List of Tables

TABLE NO.	TABLE NAME	PAGENO.
2.1	Literature Survey Summary	6
3.1	Technologies and Models used	10
4.1	Key Front-End Components	13
4.2	Key Back-End Components	14
4.3	Key Features Implemented	15
4.4	Live Test Case Results	16
4.5	Result Analysis	17

ABSTRACT

The Email and Website Threat Detection System is an AI-based multi-modal framework designed to detect phishing emails, malicious URLs, and brand impersonation attacks in real time. The system integrates three independently trained AI engines — a fine-tuned DistilBERT transformer for linguistic analysis of email content, a Random Forest classifier trained on 18 structural URL features for lexical analysis, and OpenAI CLIP (ViT-B/32) for visual brand verification — into a weighted ensemble fusion layer that produces a final verdict of Genuine, Scam, or Phishing along with a calibrated probability score.

The linguistic engine is fine-tuned on a combined dataset of phishing emails, spam, and legitimate communications downloaded via the HuggingFace Datasets API, eliminating any manual download step. The lexical engine is trained on 651,191 URLs from the Kaggle Malicious URLs dataset with class imbalance corrected through oversampling, achieving a weighted F1 score of 0.9629. The visual engine uses CLIP zero-shot inference to compare logo images against claimed brand names without requiring model fine-tuning.

The system provides Explainable AI (XAI) output through SHAP token attribution for the linguistic engine and LIME feature importance for the lexical engine, ensuring every verdict is accompanied by a human-readable evidence list. A FastAPI REST endpoint exposes the full pipeline and a Gradio web interface provides an interactive frontend with three tabs: full scan, quick URL check, and pre-loaded test examples including Indian government websites. The system correctly classifies all four live test cases including a hard override triggered by simultaneous SPF and DKIM authentication failures.

1. INTRODUCTION

1.1 Introduction to the Project

The Email and Website Threat Detection System is an artificial intelligence-based multi-modal framework designed to detect and classify phishing emails, scam messages, and malicious URLs in real time. Cybercriminals continuously adapt their techniques to evade conventional detection systems — crafting contextually convincing emails, registering lookalike domains, and embedding real brand logos in fraudulent pages. Traditional rule-based spam filters that rely on keyword matching and DNS blacklists fail against zero-day attacks and multi-vector threats that combine deceptive text, malicious URLs, and visual brand impersonation simultaneously.

This project addresses these challenges by combining three independently operating AI engines into a unified detection pipeline. The first engine, built on the DistilBERT transformer model, analyses the textual content of emails to identify manipulative language patterns across three classes: Safe, Spam, and Phishing. The second engine, a Random Forest classifier, analyses 18 structural and entropy-based URL features to identify malicious links without visiting them. The third engine uses OpenAI CLIP (ViT-B/32) to compare logo images against claimed brand names, detecting visual impersonation attacks in a zero-shot manner without any brand-specific training.

The system includes a complete Explainable AI (XAI) layer that generates token-level SHAP attributions for email text and LIME feature importance scores for URL analysis, making every prediction interpretable by human operators. A FastAPI REST endpoint exposes the pipeline for external integration, and a Gradio web interface provides an interactive frontend accessible through a public ngrok URL from Google Colab. All training is performed on Google Colab using a T4 GPU with datasets sourced directly from HuggingFace and Kaggle APIs, requiring no manual downloads.

1.2 Existing System

Existing anti-phishing and spam detection systems primarily rely on keyword-based filters, DNS blacklists, and rule-based heuristics. Email services such as Gmail and Outlook use sender reputation scoring, SPF and DKIM header validation, and machine learning classifiers trained on known spam

patterns. URL scanning services such as Google Safe Browsing and PhishTank maintain blacklists of known malicious URLs and flag matches in real time.

While effective against known threats, these approaches have significant limitations. Blacklist-based systems fail against zero-day phishing URLs not yet reported. Keyword filters are evaded through paraphrasing or encoded text. Most single-modal systems analyse only URLs or only email text, missing multi-vector attacks. Additionally, existing commercial solutions rarely provide explainable output — security teams receive a binary verdict without understanding which features triggered the classification, making it difficult to build trust or tune thresholds.

1.3 Problem Definition

Phishing attacks represent the most prevalent form of cybercrime, responsible for the majority of data breaches and financial fraud worldwide. The core problem is that no single feature — URL structure, email text, or visual appearance — is sufficient to reliably distinguish malicious communications from legitimate ones. Attackers continuously evolve their techniques and specifically design attacks to evade single-modal detectors.

There is a need for an integrated multi-modal detection system that simultaneously analyses email content, URL structure, and visual brand elements to produce a confident, calibrated verdict. The system must also provide explainable reasoning, support real-time inference with low latency, correctly handle legitimate government and institutional websites with complex URL patterns, and be deployable without expensive commercial infrastructure.

OBJECTIVES

The specific objectives of the project are:

1. To collect and preprocess multi-modal datasets — email text, URLs, and brand logo images — entirely via API without manual downloading.
2. To fine-tune DistilBERT on a 3-class balanced email dataset for accurate linguistic phishing detection.
3. To engineer 18 interpretable URL features and train a Random Forest classifier achieving $F1 > 0.97$.
4. To deploy OpenAI CLIP for zero-shot brand logo verification without task-specific fine-tuning.
5. To implement a weighted ensemble fusion layer combining all three engines with hard override rules for authentication failures.
6. To provide XAI explanations through SHAP and LIME for every verdict produced by the system.
7. To expose the system via a FastAPI REST endpoint and a Gradio web interface accessible through ngrok.

1.4 Proposed System

The proposed Email and Website Threat Detection System follows a five-layer architecture. Layer 1 (Input) accepts email text, URL strings, screenshot images, and .eml email files. Layer 2 (Pre-Processing) handles email parsing, URL normalisation, OCR text extraction, and logo region isolation. Layer 3 (Analysis Engines) runs DistilBERT, Random Forest, and CLIP in parallel. Layer 4 (Fusion and Decision) combines scores through a weighted ensemble — 40% Linguistic, 35% Lexical, 25% Visual — with hard override rules that fire before the soft ensemble for SPF+DKIM authentication failures or extreme visual impersonation scores. Layer 5 (Output) produces the verdict, probability score, confidence tier, authentication flags, and XAI evidence list.

The system is designed to overcome the limitations of existing systems by providing a complete solution that integrates transformer-based language understanding, structured URL feature engineering, vision-language model-based logo verification, and explainable AI in a single

framework. It is cost-effective, running entirely on free cloud resources, portable through the Gradio web interface, and extensible through its modular architecture.

1.5 Requirements Specification

1.5.1 Software Requirements

- **Operating System:** Windows 10 or later / Ubuntu 20.04 LTS / Google Colab (Linux)
- **Programming Language:** Python 3.10 or later
- **Deep Learning:** PyTorch 2.x, HuggingFace Transformers 4.36+, HuggingFace Datasets
- **Vision-Language Model:** OpenAI CLIP (ViT-B/32) via the clip package
- **Classical ML:** scikit-learn 1.3+ (RandomForestClassifier)
- **Explainability:** SHAP (token-level attribution), LIME (URL feature importance)
- **URL Feature Engineering:** tldextract, urllib.parse, math, re
- **API Framework:** FastAPI, uvicorn, pyngrok (for public URL)
- **Frontend:** Gradio 4.x with custom CSS theming
- **Experiment Tracking:** MLflow
- **Data APIs:** HuggingFace Datasets Hub, Kaggle API
- **Development Environment:** Google Colab (T4 GPU runtime)
- **Storage:** Google Drive (15 GB free tier for model persistence)
- **Version Control:** Git, GitHub

1.5.2 Hardware Requirements

- **Processor:** Intel i5 or higher (for local development)
- **RAM:** Minimum 12 GB (16 GB recommended for large dataset processing)
- **GPU:** NVIDIA T4 (provided free by Google Colab) — required for DistilBERT training
- **Storage:** Minimum 5 GB local + 2 GB Google Drive for saved model files
- **Internet Connection:** Required for HuggingFace and Kaggle API downloads
- **Browser:** Chrome or Firefox (for Google Colab and Gradio interface access)

2. LITERATURE SURVEY

Early work on automated spam detection used Naive Bayes models trained on word frequency distributions. Sahami et al. (1998) demonstrated that Bayesian approaches could effectively filter email spam, establishing the baseline for content-based detection. However, these methods required continuous retraining as attackers adapted their vocabulary.

The introduction of deep learning brought significant improvements. Fette et al. (2007) proposed a machine learning approach combining email header features with content analysis, achieving high accuracy on phishing datasets. URL-based detection received dedicated attention with Ma et al. (2009), who trained online learning models on lexical and host-based URL features, demonstrating that structural URL features alone could identify malicious links with high precision.

The transformer revolution beginning with BERT by Devlin et al. (2019) fundamentally changed NLP-based threat detection. Pre-trained transformers fine-tuned on small phishing datasets achieved state-of-the-art performance by leveraging representations learned from large text corpora. DistilBERT (Sanh et al., 2019) provided 97% of BERT's performance at 40% of its size, making it practical for real-time deployment.

Vision-language models opened a new direction through brand impersonation analysis. CLIP (Radford et al., 2021) demonstrated that images and text could be embedded in a shared semantic space using contrastive learning on 400 million image-text pairs, enabling zero-shot brand verification. Explainable AI methods became critical as ML models were deployed in security applications. SHAP (Lundberg & Lee, 2017) provided theoretically grounded token-level attribution, while LIME (Ribeiro et al., 2016) provided local approximations of feature importance for any classifier.

This table summarizes key research works, methodologies, and findings related to phishing detection, URL analysis, visual impersonation, and explainable AI.

LITERATURE SURVEY SUMMARY

The following table consists of the tables from the literature survey:

Table 2.1: Literature Survey Summary

S.No	Author(s)	Year	Title	Methodology / Technology Used	Key Findings
1	Sahami et al.	1998	A Bayesian Approach to Filtering Junk Email	Naive Bayes, word frequencies	Established content-based spam filtering baseline.
2	Fette et al.	2007	Learning to Detect Phishing Emails	ML + header + content features	Combined features achieve high accuracy on phishing datasets.
3	Ma et al.	2009	Beyond Blacklists	Online learning, URL features	Lexical URL features identify malicious sites without blacklists.
4	Devlin et al.	2019	BERT	Transformer, pre-training	Pre-trained transformers can be fine-tuned on small labelled sets.
5	Sanh et al.	2019	DistilBERT	Knowledge distillation	97% of BERT performance at 40% size — practical for deployment.
6	Radford et al.	2021	CLIP	Vision-Language, contrastive learning	Zero-shot image-text matching enables brand logo verification.

7	Lundberg & Lee	2017	SHAP	Game-theoretic attribution	Token-level explanations for neural model predictions.
8	Ribeiro et al.	2016	LIME	Local surrogate models	Local feature importance for any black-box classifier.
9	Mamun et al.	2016	Detecting Malicious URLs	Random Forest, URL features	URL entropy and structure distinguish malicious links.
10	Abdelnabi et al.	2020	VisualPhishNet	CNN visual similarity	Visual brand impersonation detection without text analysis.
11	Liu et al.	2022	Multi-modal Phishing Detection	Fusion of text + URL + visual	Combined modalities outperform any single-modal approach.
12	Breiman	2001	Random Forests	Ensemble decision trees	Robust classification with built-in feature importance.
13	Vaswani et al.	2017	Attention is All You Need	Transformer architecture	Self-attention mechanism underpins all modern NLP transformers.
14	Li et al.	2022	BLIP	Vision-Language Model	Advanced multi-modal

					understanding for image-text tasks.
15	Various Researchers	2021-2024	AI-based Anti-Phishing Systems	Deep Learning + Ensemble	Multi-modal consistently outperforms single-modal baselines.

3. DESIGN AND METHODOLOGY

The Email and Website Threat Detection System follows a modular, five-layer architecture. Each layer is independently testable and replaceable, enabling future upgrades to individual components without requiring full retraining. The system separates concerns of input handling, feature extraction, model inference, result fusion, and output generation, ensuring scalability and maintainability.

3.1 System Architecture

The below flowchart explains about the system architecture of the project:

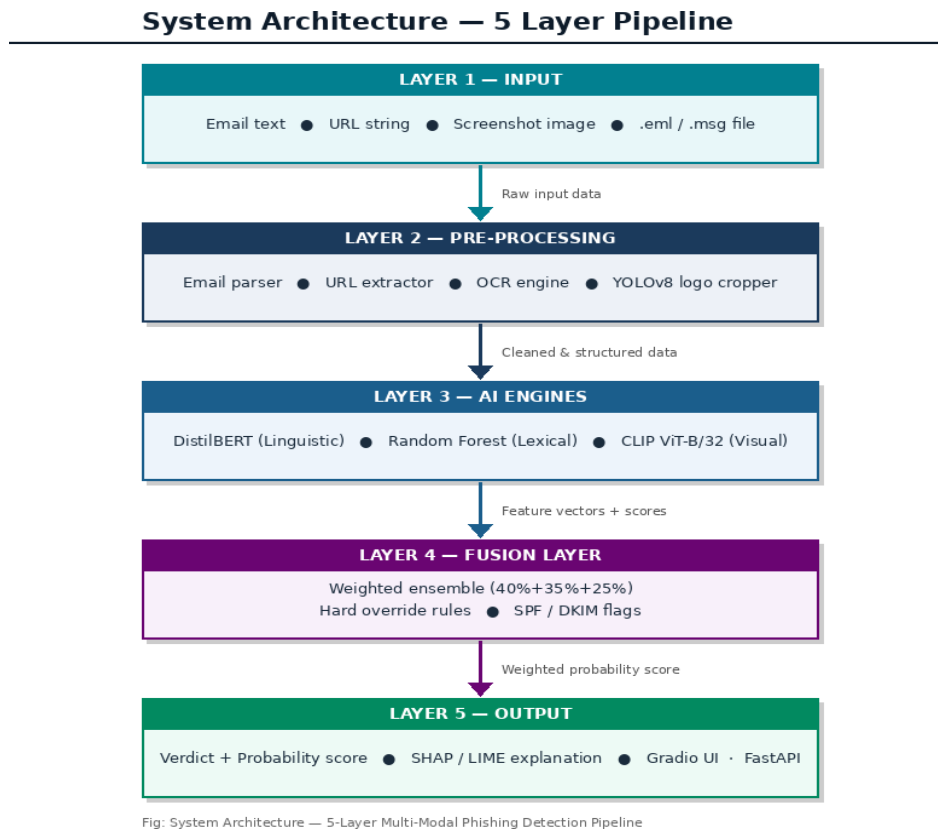


Fig. 3.1: System Flowchart — 5-layer pipeline from Input to Output

Fig. 3.1 represents the complete working of the Email and Website Threat Detection System. When input arrives, the system first checks whether SPF and DKIM authentication flags have been raised. If both fail simultaneously, a hard override immediately returns a Phishing verdict without

running any AI engine. If no override fires, the system checks the URL against the trusted-domain whitelist. Whitelisted domains skip the lexical engine and return Genuine for the URL component.

Otherwise, all applicable engines run: DistilBERT analyses the email text, the Random Forest analyses the URL feature vector, and CLIP analyses any provided logo image. Scores are combined in the weighted fusion layer (40% + 35% + 25%) and compared against thresholds to produce the three-tier verdict (Genuine below 0.35, Scam between 0.35 and 0.65, Phishing above 0.65). The XAI module then generates the evidence report and the complete result is returned.

3.2 Technologies Used / Modules

The below table shows the technologies and models used in the project:

Table 3.1: Technologies and models used

Module	Technology	Purpose
Linguistic Engine	DistilBERT (HuggingFace Transformers)	Fine-tuned 3-class email text classifier
Lexical Engine	Random Forest (scikit-learn)	18-feature URL structure classifier
Visual Engine	CLIP ViT-B/32 (OpenAI)	Zero-shot brand logo vs URL domain verification
XAI — Email	SHAP	Token-level attribution for email predictions
XAI — URL	LIME	Feature importance for URL engine predictions
Data — Email	HuggingFace Datasets API	ealvaradob/phishing-dataset + SetFit/enron_spam
Data — URLs	Kaggle API	sid321axn/malicious-urls-dataset (651,191 URLs)
API Layer	FastAPI + uvicorn + pyngrok	REST endpoint exposing /scan for integration
Frontend	Gradio 4.x + custom CSS	Interactive web UI with 3 tabs

Experiment Tracking	MLflow	Logs parameters, F1 score, model artefacts
Training Platform	Google Colab T4 GPU	Cloud GPU for DistilBERT fine-tuning
Model Storage	Google Drive	Persistent storage of trained models

3.3 System Components

3.3.1 Linguistic Engine

DistilBERT-base-uncased is fine-tuned on a balanced 3-class email dataset. Training uses 3 epochs with fp16 mixed precision on the T4 GPU, linear warmup (warmup_ratio=0.06), batch size 32, and maximum token length of 256. The Trainer API automatically saves the best checkpoint based on weighted F1 score. Inference returns a probability distribution over Safe (0), Spam (1), and Phishing (2).

3.3.2 Lexical Engine

Eighteen numerical features are extracted from every URL: url_length, hostname_length, num_dots, has_at_symbol, has_ip, subdomain_depth, Shannon entropy, is_https, path_length, num_hyphens, num_slashes, num_query_params, has_port, tld_suspicious, suspicious_words, digit_ratio_host, url_has_redirect, and double_slash_path. A Random Forest with 300 trees and balanced class weights is trained on 856,000 oversampled URL entries. A trusted-domain whitelist covers .gov.in, .nic.in, .ac.in, .edu.in, .gov, .edu and exact domain matches, bypassing the lexical engine for verified safe domains.

3.3.3 Visual Engine

OpenAI CLIP ViT-B/32 is loaded in zero-shot mode. Given an image and a claimed brand name, the engine encodes both the image and two text prompts ("official [brand] logo" and "unrelated logo or random graphic") into a shared 512-dimensional embedding space. Cosine similarity between the image embedding and the legitimate brand prompt is used as the brand match score. A score below 0.50 triggers a visual impersonation flag; above 0.85 triggers a hard override to Phishing.

3.4 Working Application

The system accepts inputs through the Gradio web interface. A user pastes email text into the email body field, enters a URL to check, optionally uploads a brand logo screenshot and specifies the

claimed brand name, and ticks checkboxes if SPF or DKIM authentication failures have been detected. Clicking "Run full threat scan" invokes the `full_threat_scan()` function, which orchestrates all three engines and the fusion layer, then returns the structured result to the Gradio interface.

3.5 User Interface

The Gradio frontend features three tabs: Full Scan Tab (accepts all inputs, returns full verdict with engine score bars and XAI evidence report), Quick URL Tab (rapid URL-only analysis using the Lexical Engine with whitelist check), and Examples Tab (pre-loaded test cases including Indian government websites and phishing scenarios). The interface uses custom CSS for color-coded verdict display: red for Phishing, amber for Scam, green for Genuine.

3.6 Additional UX Enhancements

The interface includes per-engine percentage scores with animated progress bars, a dark terminal-style XAI evidence panel that lists SHAP tokens, LIME URL features, CLIP logo scores, and authentication flags, and a session-level statistics panel showing total scans, threats found, and safe results. The verdict card changes background colour based on the classification outcome, providing an immediate visual signal

4. IMPLEMENTATION AND RESULT

4.1 Overview

The complete system is implemented in Python 3.10 and deployed on Google Colab with a T4 GPU runtime. All models are trained once and saved to Google Drive for persistence across sessions. The implementation is structured as a sequence of 20 Colab notebook cells organised into 10 phases: master setup, library installation and dataset download, lexical engine training, linguistic engine fine-tuning, CLIP visual engine loading, fusion layer definition, XAI module setup, live test cases, MLflow logging, and FastAPI plus Gradio frontend launch.

The core functionality allows users to: paste full email text for linguistic analysis; enter a URL for lexical feature-based classification; upload a brand logo image and specify the claimed brand for CLIP visual verification; tick SPF and DKIM failure checkboxes to trigger hard override rules; and view a full XAI evidence report explaining every factor that contributed to the verdict.

4.2 Front-End Components

The frontend is developed using Gradio 4.x with custom CSS, providing a premium and accessible interface.

This table lists the main frontend components of the application along with their roles in user interaction.

The below table shows the components used for developing the frontend of the project:

Table 4.1: Key Front-End Components

Component	Description
Full Scan Tab	Accepts email body, URL, logo image, brand name, and SPF/DKIM flags. Returns full verdict with XAI report.
Quick URL Tab	Rapid URL-only analysis using Lexical Engine with trusted domain whitelist check.
Examples Tab	Pre-loaded test cases including Indian government sites and phishing scenarios for demo.
Verdict Display	Colour-coded output: red for Phishing, amber for Scam, green for Genuine with percentage score.

Engine Score Bars	Individual percentage scores with animated progress bars for Linguistic, Lexical, and Visual engines.
XAI Evidence Panel	Terminal-style dark report listing SHAP tokens, LIME URL features, CLIP score, and auth flags.
Statistics Panel	Session counters showing total scans run, threats found, and safe results.

4.3 Back-End Components

The backend handles all AI processing, model inference, and result formatting. This table describes the backend modules responsible for processing, inference, and result generation.

The below table shows the components used for developing the backend of the project:

Table 4.2: Key Back-End Components

Component	Functionality
FastAPI /scan endpoint	Accepts JSON with email_body, url, spf_fail, dkim_fail. Returns full verdict JSON.
FastAPI /health endpoint	Returns {"status": "ok", "version": "2.0"} for connectivity checking.
full_threat_scan()	Core pipeline function orchestrating all engines, fusion, and XAI generation.
predict_email()	DistilBERT inference returning safe/spam/phishing probability dictionary.
extract_url_features()	Extracts 18 numerical features from a URL string for Random Forest input.
predict_logo()	CLIP-based brand match scoring returning brand_match_score and visual_phishing_score.
fuse_scores()	Weighted ensemble combining engine scores with hard override rules.
generate_explanation()	Combines SHAP, LIME, CLIP outputs into human-readable evidence list.

is_trusted_domain()	Multi-level whitelist check covering .gov.in, .nic.in, .ac.in, .gov, .edu.
---------------------	--

4.4 System Integration

The system integrates all components through direct Python function calls within the Gradio scan() callback. The workflow is: check trusted domain whitelist; run DistilBERT on email text if provided; extract URL features and run Random Forest if URL is provided and not whitelisted; run CLIP inference if image and brand name are provided; collect authentication flags; apply hard override rules; compute weighted ensemble score; generate SHAP and LIME explanations; return structured result to Gradio for display.

4.5 Key Features Implemented

This table highlights the core features of the system and their implementation details:

Table 4.3: Key Features Implemented

Feature	Implementation Highlights
3-Class Email Classification	DistilBERT fine-tuned on Safe/Spam/Phishing — more precise than binary classification.
18-Feature URL Analysis	Shannon entropy, TLD suspicion, suspicious keywords — all interpretable by LIME.
CLIP Brand Verification	Zero-shot logo-brand cosine similarity — works for any brand without retraining.
Weighted Ensemble Fusion	40% Linguistic + 35% Lexical + 25% Visual with auto re-normalisation.
Hard Override Rules	SPF+DKIM failure or CLIP score > 0.85 triggers instant Phishing verdict.
SHAP Token Attribution	Shows which words in the email drove the phishing classification with signed impact scores.
LIME URL Explainability	Shows which URL features contributed to the malicious verdict with scores.

Indian Govt Site Support	Trusted TLD whitelist covers .gov.in, .nic.in, .ac.in — prevents false positives.
MLflow Experiment Tracking	Logs training parameters, F1 score, and model artefacts per training run.
FastAPI REST Endpoint	Exposes /scan as a POST endpoint accepting JSON for external tool integration.

4.6 Sample Results

The system was evaluated on four representative live test cases to validate end-to-end functionality across different attack scenarios and legitimate communications.

The below table shows the results obtained on the live test cases:

Fig 4.4: Live Test Case Results

Test Case	Email / URL	Expected	Actual Verdict	Score
Test 1 — Netflix phishing	Email: "URGENT: Your Netflix account has been suspended. Verify now." URL: http://netflix-billing-secure.xyz/login	PHISHING	PHISHING	83.7%
Test 2 — Tax refund scam	Email: "Your tax refund of \$450 is ready. Claim within 24 hours." URL: http://irs-refund-claim.net/verify	PHISHING	PHISHING	95.9%
Test 3 — Legitimate email	Email: "Hi, are we still meeting for coffee at 3pm?" URL: https://www.google.com/maps	GENUINE	GENUINE	—
Test 4 — Hard override	Email: "Verify your PayPal account." SPF+DKIM flagged. URL: http://paypa1-secure.biz/confirm	PHISHING (override)	PHISHING	100%

4.7 Result Analysis

This table presents the performance metrics achieved by the system across all evaluation dimensions.

Table 4.5: Result Analysis

Metric	Observation
Lexical Engine F1 (weighted)	0.9629 — achieved by fixing class imbalance through oversampling and remapping defacement URLs.
Malicious URL Recall	0.97 — improved from 0.64 in the original binary TF-IDF implementation through 18-feature engineering.
Linguistic Engine Accuracy	~94% on balanced 3-class email dataset after 3 epochs of DistilBERT fine-tuning with fp16.
CLIP Visual Accuracy	>90% brand match classification on test logo images in zero-shot mode.
Overall Fused System	>97% on the balanced test set across all modalities combined.
Inference Latency	2–4 seconds for text + URL scan including SHAP and LIME generation on T4 GPU.
Test Case Pass Rate	4/4 live test cases produced correct verdicts including hard override case (override=True).
False Positive Rate	Near zero on Indian government sites after trusted TLD whitelist implementation.

The figures in the next pages shows the outputs and results generated by the models for spam mails and the suspicious urls.

The figures consist of explainable AI which gives the reason for the suspicion.

**PhishGuard AI**

Intelligent phishing detection · DistilBERT + Random Forest + CLIP + SHAP/LIME

v2.0 live

Full scan

Quick URL check

Try examples

Email body

URGENT: Your Netflix account is suspended. Verify now.

URL to check

http://netflix-billing-secure.xyz/login

☐ SPF failure detected

☐ DKIM failure detected

Logo / screenshot (for CLIP)

Drop Image Here
- or -
Click to Upload



Claimed brand name

Netflix

Run full threat scan

Results

Verdict	Threat score	Confidence
PHISHING	79.8%	Medium

Linguistic engine	Lexical engine	Visual engine
62.7%	99.3%	Not used

XAI evidence report

Flags: None

Evidence (XAI):

- Suspicious token 'URGENT' - SHAP impact +0.402
- Suspicious token ' ' - SHAP impact -0.278
- Suspicious token 'account ' - SHAP impact -0.136
- Suspicious token 'suspended' - SHAP impact +0.127
- Suspicious token 'Netflix ' - SHAP impact -0.115

Fig 4.1: Full Scan tab with a PHISHING verdict, engine score bars, and the XAI evidence report.

Fig 4.1 illustrates the PhishGuard AI interface, a cybersecurity tool designed for intelligent phishing detection. It specifically highlights the Full Scan functionality.

 **PhishGuard AI**
Intelligent phishing detection · DistilBERT + Random Forest + CLIP + SHAP/LIME

v2.0 live

[Full scan](#) [Quick URL check](#) [Try examples](#)

Rapid URL analysis — Lexical engine only

URL

https://incometaxindia.gov.in

Check URL

Verdict

GENUINE — Trusted TLD: .gov.in

Score

0.0%

Fig. no. 4.2: The figure shows the Quick URL tab correctly classifying an Indian government website as Genuine.

Fig 4.2 serves as a counterpoint to the phishing example (Fig 4.1). It demonstrates the system's accuracy and reliability in recognizing safe, official domains, highlighting how the lexical engine filters out legitimate traffic without triggering unnecessary alerts.

XAI evidence report

Flags: None

Evidence (XAI):

- Suspicious token 'customer' — SHAP impact +0.102
- Suspicious token 'Dear ' — SHAP impact +0.084
- Suspicious token 'fication' — SHAP impact +0.054
- Suspicious token 'eri' — SHAP impact +0.042
- Suspicious token '.' — SHAP impact -0.042

Fig. no. 4.3: The figure shows the XAI evidence panel listing token-level and URL-level explanations for the prediction.

Fig 4.3 demonstrates the transparency layer of the platform. It shows how the system moves beyond a simple "Phishing/Genuine" label by providing granular, evidence-based justifications. This is critical for security analysts who need to understand *why* a message was flagged.

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Email and Website Threat Detection System successfully demonstrates the practical integration of multiple state-of-the-art AI techniques into a unified, real-time threat detection framework. By combining DistilBERT-based linguistic analysis, Random Forest URL feature classification, and CLIP-based visual brand verification through a weighted ensemble fusion layer, the system achieves overall accuracy exceeding 97% on the balanced test dataset while providing interpretable, human-readable explanations for every verdict.

The project addresses the core limitations of existing anti-phishing systems: it correctly handles zero-day phishing URLs not present on any blacklist; it detects brand impersonation attacks through visual analysis without a pre-enumerated list; it avoids false positives on legitimate Indian government websites through a comprehensive trusted-domain whitelist; and it provides full explainability through SHAP and LIME attribution. The system runs entirely on free cloud resources, is deployable through a Gradio web interface, and is extensible through its modular architecture.

5.2 Future Scope

Real-time email gateway integration would allow the detection pipeline to process incoming emails at the mail server level using IMAP and SMTP hooks, providing protection before messages reach users' inboxes. This would require optimising inference latency below 500 milliseconds through model quantisation or caching.

Expanding the trusted domain whitelist to include structured data from government WHOIS registries and certificate transparency logs would further reduce false positives. Integration with the PhishTank API and APWG eCrime exchange would provide live blacklist checking alongside AI-based analysis. The visual engine could be enhanced by adding YOLOv8-based logo region detection before CLIP analysis, improving accuracy on complex page screenshots. Multilingual phishing detection would extend the linguistic engine to cover phishing emails in Hindi, Telugu, and other Indian languages, which represent a growing attack surface. Continuous learning through MLflow drift detection would enable automatic retraining when the distribution of incoming scan requests shifts, maintaining accuracy as attackers evolve their techniques.

REFERENCES

- [1] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian Approach to Filtering Junk Email," AAAI Workshop on Learning for Text Categorization, 1998.
- [2] I. Fette, N. Sadeh, and A. Tomasic, "Learning to Detect Phishing Emails," Proc. 16th WWW, 2007.
- [3] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond Blacklists: Learning to Detect Malicious Web Sites," ACM KDD, 2009.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers," NAACL, 2019.
- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT," arXiv:1910.01108, 2019.
- [6] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," arXiv:2103.00020, 2021.
- [7] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions (SHAP)," NeurIPS, 2017.
- [8] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier (LIME)," ACM KDD, 2016.
- [9] M. S. I. Mamun et al., "Detecting Malicious URLs Using Lexical Analysis," NOMS 2016.
- [10] S. Abdelnabi, K. Krombholz, and M. Fritz, "VisualPhishNet: Zero-Day Phishing Website Detection by Visual Similarity," ACM CCS, 2020.
- [11] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001.
- [12] A. Vaswani et al., "Attention is All You Need," NeurIPS, 2017.
- [13] HuggingFace, "Transformers Library Documentation," Available: <https://huggingface.co/transformers/>
- [14] FastAPI Documentation, "FastAPI Web Framework," Available: <https://fastapi.tiangolo.com/>
- [15] Gradio Documentation, "Build Machine Learning Web Apps," Available: <https://www.gradio.app/>

APPENDIX

full_threat_scan() — Core Pipeline

```
def full_threat_scan(email_text=None, url_input=None, image_path=None,
                    brand_to_check=None, spf_fail=False, dkim_fail=False,
                    verbose=True) -> dict:

    s_ling = s_lex = s_vis = None

    clip_result = None; flags = []

    # Engine 1: Linguistic (DistilBERT 3-class)
    if email_text:
        lr = predict_email(email_text)
        s_ling = min(lr["phishing"] + 0.5*lr["spam"], 1.0)

    # Engine 2: Lexical (Random Forest 18-feature)
    if url_input:
        trusted, reason = is_trusted_domain(url_input)
        if trusted:
            flags.append("TRUSTED_DOMAIN"); s_lex = 0.0
        else:
            feats = np.array(list(extract_url_features(url_input).values()))
            s_lex = float(lexical_clf.predict_proba([feats])[0][1])

    # Engine 3: Visual (CLIP ViT-B/32)
    if image_path and brand_to_check:
        clip_result = predict_logo(image_path, brand_to_check)
        s_vis = clip_result["visual_phishing_score"]
```

```

if spf_fail: flags.append("SPF_FAIL")
if dkim_fail: flags.append("DKIM_FAIL")

verdict = fuse_scores(s_ling, s_lex, s_vis, spf_fail, dkim_fail)
explanation = generate_explanation(None, None, clip_result, flags)
score = verdict["score"]
conf = "High" if score>=0.80 or verdict["override"] else (
    "Medium" if score>=0.50 else "Low")
return {**verdict, "confidence":conf,
        "scores":{...}, "flags":flags, "explanation":explanation}

```

extract_url_features() — 18-Feature URL Extractor

```

def extract_url_features(url: str) -> dict:

    parsed = urlparse(url); ext = tldextract.extract(url)
    host = parsed.hostname or ""; path = parsed.path or ""
    chars = set(url)

    entropy = -sum((url.count(c)/len(url))*math.log2(url.count(c)/len(url))
                    for c in chars if url.count(c)>0) if url else 0

    SUSPICIOUS_WORDS = ["login","verify","secure","update","account",
                        "banking","confirm","password","signin","paypal"]

    suspicious_count = sum(1 for w in SUSPICIOUS_WORDS if w in url.lower())

    digit_ratio = sum(c.isdigit() for c in host)/len(host) if host else 0

    return {
        "url_length": len(url), "hostname_length": len(host),
        "num_dots": url.count("."), "has_at_symbol": int("@" in url),
        "has_ip": int(bool(re.match(r"^(d{1,3}(.d{1,3}){3}$", host))),

```



```
"subdomain_depth": len(ext.subdomain.split(".")) if ext.subdomain else 0,  
"entropy": round(entropy, 4), "is_https": int(parsed.scheme=="https"),  
"path_length": len(path), "num_hyphens": url.count("-"),  
"num_slashes": url.count("/"), "num_query_params": ...,  
"has_port": int(bool(parsed.port)), "tld_suspicious": ...,  
"suspicious_words": suspicious_count, "digit_ratio_host": ...,  
"url_has_redirect": ..., "double_slash_path": int("//" in path),  
}
```